Python for AI & Machine Learning

Class 13 – Encapsulation in Python

Trainer: Surya Dekshith Mallikarjuna
Course: Python for AI & Machine Learning
Duration: 2 Hours (Theory + Practical)

Learning Objectives

After completing this session, students will be able to:

- Understand the concept of Encapsulation.
- Explain why Encapsulation is one of the four pillars of Object-Oriented Programming.
- Differentiate between Public, Protected, and Private members.
- Understand Data Hiding.
- Create Getter and Setter methods.
- Protect sensitive data inside a class.
- Implement Encapsulation in real-world and AI & Machine Learning applications.

Revision of Previous Class

In the previous class, we learned:

- Instance Variables
- Class Variables
- Instance Methods
- Class Methods
- Static Methods

Today, we begin learning the **Four Pillars of Object-Oriented Programming (OOP).**

The four pillars are:

1. Encapsulation ✅ (Today's Topic)
2. Inheritance
3. Polymorphism
4. Abstraction

These concepts help developers build secure, reusable, and maintainable software.

What is Encapsulation?

Encapsulation is the process of **combining data (variables) and methods (functions) into a single unit called a class while restricting direct access to the data.**

In simple terms:

> **Encapsulation means protecting an object's data by allowing access only through approved methods.**

This technique is also known as **Data Hiding**.

Why Do We Need Encapsulation?

Without Encapsulation:

- Anyone can modify important data.
- Programs become difficult to maintain.
- Data integrity cannot be guaranteed.
- Sensitive information can be exposed.

With Encapsulation:

- Data is protected.
- Access is controlled.
- Validation rules can be applied.
- Programs become safer and more reliable.

Real-Life Example – ATM Machine

Think about an ATM.

A customer can:

- Insert an ATM card.
- Enter a PIN.
- Check account balance.
- Withdraw cash.
- Deposit money.

However, the customer **cannot directly access or modify the bank's database**.

The ATM provides only approved operations.

This is an example of **Encapsulation**.

Another Real-Life Example – Hospital Management System

A patient's medical records contain sensitive information.

- Doctors can update records.
- Patients can view permitted information.
- Other users cannot directly modify the records.

Encapsulation helps keep patient information secure.

Access Specifiers in Python

Python follows naming conventions to indicate access levels.

Access Type	Syntax	Description
Public	`name`	Accessible from anywhere
Protected	`_name`	Intended for internal use and subclasses
Private	`__name`	Intended for use only inside the class

Public Members

Public members can be accessed from any part of the program.

Example

```python id="sg6v84" class Student:

```
def __init__(self):
    self.name = "Rahul"
```

student = Student()

print(student.name)

```
Output
```
```

```
``` id="agk2i1"
Rahul
```

Public members are suitable for information that does not require protection.

Protected Members

Protected members begin with a single underscore (`_`).

They are intended for use within the class and subclasses.

Example

```python id="fs0r7s" class Student:

```
def __init__(self):
    self._course = "AI & ML"
```

student = Student()

print(student._course)

```
### Output

``` id="x4ud8q"
AI & ML
```

Although they can still be accessed, developers treat them as internal members.

Private Members

Private members begin with a double underscore (`__`).

Python applies **name mangling**, making direct access from outside the class difficult.

Example

```python id="8v0s5q" class Employee:

```
def __init__(self):
    self.__salary = 50000
```

employee = Employee()

print(employee.__salary)

```
### Output

``` id="hkjlwm"
AttributeError:
'Employee' object has no attribute '__salary'
```

Private members should be accessed only through methods defined inside the class.

Accessing Private Members

Instead of accessing private variables directly, we use methods.

Example

```python id="g8ckxt" class Employee:

```
def __init__(self):
    self.__salary = 50000

def show_salary(self):
    print("Salary:", self.__salary)
```

employee = Employee()

employee.show_salary()

```
### Output

``` id="eh3df0"
Salary: 50000
```

Getter Methods

A Getter method returns the value of a private variable.

Example

```python id="fv56ii" class Employee:

```
def __init__(self):
    self.__salary = 50000

def get_salary(self):
    return self.__salary
```

employee = Employee()

print(employee.get_salary())

```
### Output

``` id="5ly5d4"
50000
```

Setter Methods

A Setter method updates a private variable after checking whether the new value is valid.

Example

```python id="uf8a0i" class Employee:

```
def __init__(self):
    self.__salary = 50000

def set_salary(self, salary):

    if salary > 0:
        self.__salary = salary
    else:
        print("Invalid Salary")

def get_salary(self):
    return self.__salary
```

employee = Employee()

employee.set_salary(60000)

print(employee.get_salary())

```
### Output
```

```
``` id="y2dtm3"
60000
```

AI Example – Machine Learning Model

Suppose a machine learning model stores its accuracy.

Accuracy should always be between **0 and 100**.

```python id="es57mu" class MLModel:

```
def __init__(self):
    self.__accuracy = 0

def train(self, accuracy):

    if 0 <= accuracy <= 100:
        self.__accuracy = accuracy
    else:
        print("Invalid Accuracy")

def display(self):
    print("Model Accuracy:", self.__accuracy, "%")
```

model = MLModel()

model.train(96)

model.display()

```
### Output

``` id="a2qq8v"
Model Accuracy: 96 %
```

Encapsulation ensures that invalid values cannot be stored.

AI Example – Student Performance

```python id="zw41ix" class AIStudent:

```
def __init__(self, name):
    self.name = name
    self.__marks = 0

def set_marks(self, marks):

    if 0 <= marks <= 100:
        self.__marks = marks
    else:
        print("Invalid Marks")

def display(self):
    print("Student:", self.name)
    print("Marks:", self.__marks)
```

student = AIStudent("Surya")

student.set_marks(95)

student.display()

```
---

# Real-World Example – Bank Account

```python id="s62fnj"
class BankAccount:

    def __init__(self, account_holder, balance):
        self.account_holder = account_holder
        self.__balance = balance

    def deposit(self, amount):

        if amount > 0:
            self.__balance += amount

    def withdraw(self, amount):

        if amount <= self.__balance:
            self.__balance -= amount
        else:
            print("Insufficient Balance")

    def check_balance(self):
        print("Current Balance:", self.__balance)

account = BankAccount("Rahul", 10000)
```
```

```
account.deposit(5000)
account.withdraw(3000)

account.check_balance()
```

Output

`id="x91nyq"`
`Current Balance: 12000`

Advantages of Encapsulation

- Protects sensitive information.
- Prevents unauthorized modification.
- Improves data security.
- Supports data validation.
- Makes code easier to maintain.
- Improves software reliability.
- Encourages modular programming.
- Reduces errors in large applications.

Applications of Encapsulation

Encapsulation is widely used in:

- Banking Systems
- Hospital Management Systems
- School Management Software
- Human Resource Management Systems (HRMS)
- Customer Relationship Management (CRM)
- AI and Machine Learning Applications
- E-commerce Platforms
- Enterprise Software

Summary

In today's session, we learned:

- Introduction to Encapsulation
- Data Hiding
- Public Members
- Protected Members
- Private Members
- Getter Methods

- Setter Methods
- AI-based examples
- Banking application example
- Importance of Encapsulation in software development

Encapsulation is essential for developing secure, maintainable, and professional software systems.

Practical Exercises

Practical 1 – Student Class

Create a `Student` class with:

Private Variable:

- `__marks`

Methods:

- `set_marks()`
- `get_marks()`

Accept only marks between **0 and 100**.

Practical 2 – Employee Class

Create an `Employee` class.

Private Variables:

- Employee ID
- Salary

Methods:

- Display Employee Details
- Update Salary

Validation:

- Salary cannot be negative.

Practical 3 – Hospital Patient

Create a `HospitalPatient` class.

Private Variables:

- Patient ID
- Medical History

Methods:

- Add Medical Record
- View Medical Record

Medical history must not be directly accessible from outside the class.

Practical 4 – AI Model

Create an `AIModel` class.

Private Variables:

- Model Name
- Accuracy

Methods:

- Train Model
- Display Model Details

Ensure that the accuracy value is between **0 and 100**.

Assignment

Develop a **Bank Management System** using Encapsulation.

Create a `BankAccount` class with the following:

Private Variables

- Account Number
- Balance

Public Methods

- Deposit Money
- Withdraw Money
- Check Balance

Requirements:

- Do not allow negative deposits.

- Prevent withdrawals greater than the available balance.
- Display appropriate messages for invalid transactions.
- Demonstrate the program using at least three customer accounts.

Interview Questions

1. What is Encapsulation?
2. Why is Encapsulation important?
3. What is Data Hiding?
4. What is the difference between Public, Protected, and Private members?
5. What is a Getter method?
6. What is a Setter method?
7. Why are private variables used?
8. Give a real-world example of Encapsulation.
9. How does Encapsulation improve software security?
10. How is Encapsulation used in AI and Machine Learning applications?

Next Class Preview

In the next session, we will study **Inheritance**, the second pillar of Object-Oriented Programming.

Topics include:

- Introduction to Inheritance
- Parent and Child Classes
- Single Inheritance
- Multiple Inheritance
- Multilevel Inheritance
- Hierarchical Inheritance
- Hybrid Inheritance
- The `super()` Function
- AI & Machine Learning examples using Inheritance